KTH – School of Engineering Sciences (SCI)

SF1546 Numerical Methods, Basic Course

Heat Transfer: From Theory to Numerical Solutions

Project VT2026

Name	**Personal ID**	**E-mail**
lastname, firstnamt	12345	`mymail@kth.se`

Group 41

April 27, 2026

Contents

1 Introduction **2**

2 Method **2**
2.1 Part 1 – Heat Exchanger . . . 2
2.1.1 Newton's Method for a Non-Linear Equation . . . 3
2.1.2 Newton's Method for a Non-linear System of Equations . . . 3
2.2 Part 2 - Heat Sinks . . . 5
2.2.1 Approximating the static heat equation . . . 5
2.2.2 Solving the System . . . 6

3 Results **6**
3.1 Part 1 – Heat Exchanger . . . 6
3.1.1 Numerical Solution of non-linear equation . . . 6
3.1.2 Numerical Solution of the non-linear system . . . 7
3.1.3 Verification for the infinite length case . . . 9
3.1.4 Method order of accuracy . . . 10
3.1.5 Non-Linear Case: SS AISI 316, Aluminium, and Copper . . . 11

4 Discussion **14**

A Python Source Code **16**

1 Introduction

In modern engineering we are often faced with challenges of optimization, whether they are about minimizing cost or maximizing efficiency each case relies on mathematical models to try and predict a systems behaviour. In the first part of this report we analyse a heat exchanger which has a range of parameters already set and attempt to find optimal geometric parameters that satisfies the specifications. We do this by showing how the problem can be converted into a *root finding problem* and solved using Newton's Method with python code. Furthermore, we verify that our solution is reasonable by comparing it to theoretical expectations of such an implementation as well as comparing results with those obtained using established libraries such as scipy.
The second part of the report is spent on modelling a one-dimensional heat sink and it's thermal diffusion properties relating to choice of material and size. We show, in a similar way to the heat exchange model, how this boundary value problem can be formulated as a system of equations and solved using finite differences approximation and Newton's Method. The results are compared to a analytical case where simplifications to the model are made such that we can find an analytical solution to the governing equation and calculate an approximate error with which we can determine the models order of accuracy and when this analytical approximation makes sense. Finally we evaluate three materials with different thermal properties and assert a optimal length and choice of material for the heat sink.

2 Method

2.1 Part 1 – Heat Exchanger

We are tasked with determining a numerical value of the *maximum* outer tube diameter d that simultaneously fulfills the specification of the other parameters given below. An equation for the heat flux is given by Equation (1).

$$Q = A_{\text{ex}} \cdot U_f \cdot \Delta T_m, \tag{1}$$

where $Q = 801368$ W is the total heat flux, $A_{\text{ex}} = 64.15$ m^2 is the heat transfer area, $\Delta T_m = 29.6$ °C is the mean temperature difference between the two fluids, and U_f is the overall heat transfer coefficient given by

$$U_f = \left(\frac{d}{D_s h_t} + \frac{dR_{fi}}{D_s} + \frac{d \ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s} \right)^{-1}. \tag{2}$$

Here, d is the outer tube diameter which is as of yet undefined and, $D_s = 1.219$ m is the inner shell diameter, R_{fi}, R_{fo}, h_s, h_t, k_w are material and thermodynamic parameters given in Table 1.

Parameter	Value	Unit
R_{fi}	$1.76 \cdot 10^{-4}$	$\frac{\text{m}^2\,°\text{C}}{\text{W}}$
R_{fo}	$1.76 \cdot 10^{-4}$	$\frac{\text{m}^2\,°\text{C}}{\text{W}}$
h_s	356	$\frac{\text{W}}{\text{m}^2\,°\text{C}}$
h_t	356	$\frac{\text{W}}{\text{m}^2\,°\text{C}}$
k_w	60	$\frac{\text{W}}{\text{m}\,°\text{C}}$

Table 1: Parameter values.

2.1.1 Newton's Method for a Non-Linear Equation

To find a numerical value for d we apply Newton's method for scalar problems programmatically as outlined in Algorithm 1.

Algorithm 1 Newton's method – scalar equation

```
Let the initial guess be d_0 and the tolerance ε
k ← 0
while |f(d_k)| > ε do
    d_{k+1} ← d_k − f(d_k)/f'(d_k)
    k ← k + 1
end while
return d_k
```

We must therefore first find our function $f(d_k)$ on which we can apply Newton's method. We begin by rearranging Equation (1) into:

$$U_f = \frac{Q}{A_{\text{ex}} \cdot \Delta T_m}, \tag{3}$$

Newton's method is a *root finding* algorithm and as such solves $f(d_k) = 0$ by definition. We therefore subtract Equation (3) from Equation (2) and arrive at our expression for $f(d_k)$:

$$f(d_k) = \left(\frac{d_k}{D_s h_t} + \frac{d_k R_{fi}}{D_s} + \frac{d_k \ln(d_k/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s} \right)^{-1} - \frac{Q}{A_{\text{ex}} \cdot \Delta T_m} = 0 \tag{4}$$

Taking the derivative with respect to d, our second term vanishes. Applying the chain rule:

$$\frac{\mathrm{d}}{\mathrm{d}d_k} f(d_k) = f'(d_k) = -\frac{\dfrac{1}{D_s h_t} + \dfrac{R_{fi}}{D_s} + \dfrac{\ln(d_k/D_s) + 1}{2k_w}}{\left(\dfrac{d_k}{D_s h_t} + \dfrac{d_k R_{fi}}{D_s} + \dfrac{d_k \ln(d_k/D_s)}{2k_w} + R_{fo} + \dfrac{1}{h_s} \right)^2} \tag{5}$$

We can now assemble the final expression for Newton's Method, the update is given by:

$$d_{k+1} = d_k - \frac{f(d_k)}{f'(d_k)} \tag{6}$$

This is implemented and solved for using the python code in Listing 1.

2.1.2 Newton's Method for a Non-linear System of Equations

We must now also consider the pressure drop ΔP and surface area of heat transfer A_{ex} as functions of the inner shell diameter D_s in our system along with the added restriction that the *maximum* ΔP allowed is 49080 Pa. We must therefore now move to solving a *system* of equations since ΔP and A_{ex} are expressed with Equation (7) and Equation (8) respectively. Table 2 give the parameter values for Equation (7) and Equation (8).

$$\Delta P = \frac{c}{D_s^2 (S_t - d)^2} \tag{7}$$

$$A_{ex} = \pi K_1 \frac{D_s^{n_1}}{d^{n_1 - 1}} \tag{8}$$

Parameter	Value	Unit
c	0.389	$\mathrm{W\,s \cdot m}$
S_t	0.016	m
K_1	0.249	m
n_1	2.207	–

Table 2: Additional parameter values.

Since we now have to solve a system of equations for both the tube diameter d as well as the inner shell diameter D_s we move to Newton's Method for a *system* of equations:

Algorithm 2 Newton's method – system

Let the initial guess be $\mathbf{x}_0 = (d_0,\, D_{s,0})^T$ and tolerance ε
$k \leftarrow 0$
while $\|\mathbf{F}(\mathbf{x}_k)\| > \varepsilon$ **do**
  Solve $J\mathbf{F}(\mathbf{x}_k)\,\boldsymbol{\delta} = -\mathbf{F}(\mathbf{x}_k)$ for $\boldsymbol{\delta}$
  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \boldsymbol{\delta}$
  $k \leftarrow k+1$
end while
return $\mathbf{x}_k = (d,\, D_s)^T$

At each iteration, the linear system $J\mathbf{F}(\mathbf{x}_k)\,\boldsymbol{\delta} = -\mathbf{F}(\mathbf{x}_k)$ is solved for the correction $\boldsymbol{\delta} = \mathbf{x}_{k+1} - \mathbf{x}_k$, which is then used to update the current guess.
We now formulate the Newton function $\mathbf{F}(d, D_s) = \mathbf{0}$ by combining Equations (1), (2), (7) and (8) into

$$F_1(d, D_s) = \left(\frac{d}{D_s h_t} + \frac{dR_{fi}}{D_s} + \frac{d\ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s}\right)^{-1} - \frac{Q\,d^{n_1-1}}{\pi K_1 D_s^{n_1}\Delta T_m} \tag{9}$$

$$F_2(d, D_s) = \frac{c}{D_s^2(S_t - d)^2} - \Delta P \tag{10}$$

Combine into our Newton vector:

$$\mathbf{F}(d, D_s) = \begin{pmatrix} F_1(d, D_s) \\ F_2(d, D_s) \end{pmatrix} = \mathbf{0}, \quad \text{with } F_1, F_2 \text{ given by Equations (9) and (10)} \tag{11}$$

To apply Newton's Method in higher dimensions we need the Jacobian $J\mathbf{F}$ which is computed analytically. Denoting $g(d, D_s) = \frac{d}{D_s h_t} + \frac{dR_{fi}}{D_s} + \frac{d\ln(d/D_s)}{2k_w} + R_{fo} + \frac{1}{h_s}$ for brevity, the entries are:

$$J\mathbf{F}(d, D_s) = \begin{pmatrix} \partial_d F_1 & \partial_{D_s} F_1 \\ \partial_d F_2 & \partial_{D_s} F_2 \end{pmatrix} = \begin{pmatrix} -\dfrac{g_d}{g^2} + \dfrac{(n_1-1)Q\,d^{n_1-2}}{\pi K_1 D_s^{n_1}\Delta T_m} & \dfrac{g_{D_s}}{g^2} + \dfrac{n_1 Q\,d^{n_1-1}}{\pi K_1 D_s^{n_1+1}\Delta T_m} \\ \dfrac{2c}{D_s^2(S_t-d)^3} & \dfrac{-2c}{D_s^3(S_t-d)^2} \end{pmatrix} \tag{12}$$

Where $g_d = \partial g/\partial d$ and $g_{D_s} = \partial g/\partial D_s$ are:

$$g_d = \frac{1}{D_s h_t} + \frac{R_{fi}}{D_s} + \frac{\ln(d/D_s) + 1}{2k_w} \tag{13}$$

$$g_{D_s} = -\frac{d}{D_s^2 h_t} - \frac{dR_{fi}}{D_s^2} - \frac{d}{2k_w D_s} \tag{14}$$

This system is then solved using the python code in Listing 2

2.2 Part 2 - Heat Sinks

For the second part we shift our focus to the dispersion of heat in a one-dimensional rod and study the governing equation, a case of the static heat equation:

$$\frac{d^2T}{dx^2} = \alpha_1(T(x) - T_\infty) + \alpha_2 f(T(x)), \quad 0 \le x \le L \tag{15}$$

where $f(t) = T^4(x) - T_\infty^4$, $\alpha_1 = \frac{4h_c}{DK}$, $\alpha_2 = \frac{4\epsilon\sigma}{DK}$ and our Dirichlet boundary conditions are $T(0) = T_s$, $T(L) = T_L$. [1]

2.2.1 Approximating the static heat equation

Now, to be able to numerically solve Equation (15) we apply the finite difference model, more specifically a central finite difference for the second derivative and solve for the temperature distribution $T(x)$ using Newtons method for a system of equations: Algorithm 2.

For some second-order differential equation:

$$\frac{d^2T}{dx^2} = \omega(x), \qquad \omega(x_j) = \omega_j, \quad x \in [0, L]$$

a second-order central finite difference approximation is given by

$$\left.\frac{d^2T}{dx^2}\right|_{x_j} \approx \frac{T_{j+1} - 2T_j + T_{j-1}}{h^2} + \mathcal{O}(h^2).$$

Where, for easier notation we let $T(x_j) = T_j$.
Divide the interval $[0, L]$ into N sub intervals and let $N = 4$, so our grid points are

$$j \in \{0, 1, 2, 3, 4\},$$

with interior points $j = 1, 2, 3$. The step size is given by

$$h = \frac{x_{end} - x_{start}}{N} = \frac{L - 0}{4} = \frac{L}{4}.$$

Therefore our points of evaluation becomes,

$$x_0 = 0, \quad x_1 = h = \frac{L}{4}, \quad x_2 = 2h = \frac{L}{2}, \quad x_3 = 3h = \frac{3L}{4}, \quad x_4 = 4h = L.$$

Now, in our case as stated in Equation (15):

$$\omega_j = \alpha_1(T(x_j) - T(\infty)) + \alpha_2 f(T(x_j)).$$

Applying the approximation on each of the interior points we obtain a system of equations:

$$\begin{cases} \dfrac{T_2 - 2T_1 + T_0}{h^2} = \alpha_1(T_1 - T(\infty)) + \alpha_2 f(T_1), \\ \dfrac{T_3 - 2T_2 + T_1}{h^2} = \alpha_1(T_2 - T(\infty)) + \alpha_2 f(T_2), \\ \dfrac{T_4 - 2T_3 + T_2}{h^2} = \alpha_1(T_3 - T(\infty)) + \alpha_2 f(T_3). \end{cases}$$

applying the boundary conditions and moving the known boundaries to the right hand-side our system takes the form:

$$\begin{cases} \dfrac{T_2 - 2T_1}{h^2} = \alpha_1(T_1 - T(\infty)) + \alpha_2 f(T_1) - \dfrac{T_s}{h^2}, \\ \dfrac{T_3 - 2T_2 + T_1}{h^2} = \alpha_1(T_2 - T(\infty)) + \alpha_2 f(T_2), \\ \dfrac{T_2 - 2T_3}{h^2} = \alpha_1(T_3 - T(\infty)) + \alpha_2 f(T_3) - \dfrac{T_L}{h^2}. \end{cases}$$

[1] See Table 5 for the constants.

Rewriting the system in compact matrix form, which can be generalized for an arbitrary N:

$$A\mathbf{T} = \mathbf{r}(\mathbf{T}),$$

where

$$\mathbf{T} = \begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix}, \qquad A = \begin{bmatrix} -2 & 1 & 0 \\ 1 & -2 & 1 \\ 0 & 1 & -2 \end{bmatrix},$$

and

$$\mathbf{r}(\mathbf{T}) = \begin{bmatrix} h^2\big[\alpha_1\big(T_1 - T(\infty)\big) + \alpha_2 f(T_1)\big] - T_s \\ h^2\big[\alpha_1\big(T_2 - T(\infty)\big) + \alpha_2 f(T_2)\big] \\ h^2\big[\alpha_1\big(T_3 - T(\infty)\big) + \alpha_2 f(T_3)\big] - T_L \end{bmatrix}.$$

2.2.2 Solving the System

As for Equation (11), we apply Newton's method to our system, define the Newton function $\mathbf{F}(\mathbf{T})$:

$$\mathbf{F}(\mathbf{T}) = A\mathbf{T} - \mathbf{r}(\mathbf{T}) = \mathbf{0}.$$

Newton's method then solves

$$\mathbf{J}_F(\mathbf{T})\,\boldsymbol{\delta} = -\mathbf{F}(\mathbf{T}),$$

where the Jacobian is

$$\mathbf{J}_F = \frac{\partial \mathbf{F}}{\partial \mathbf{T}} = A - \frac{\partial \mathbf{r}}{\partial \mathbf{T}}.$$

and therefore:

$$\mathbf{J}_F = \begin{bmatrix} -2 - h^2\big(\alpha_1 + \alpha_2 f'(T_1)\big) & 1 & 0 \\ 1 & -2 - h^2\big(\alpha_1 + \alpha_2 f'(T_2)\big) & 1 \\ 0 & 1 & -2 - h^2\big(\alpha_1 + \alpha_2 f'(T_3)\big) \end{bmatrix}.$$

With the code provided in Listing 3 we then solve for the temperature distribution function $\mathbf{T}(x)$.

3 Results

3.1 Part 1 – Heat Exchanger

3.1.1 Numerical Solution of non-linear equation

The implementation of Newton's Method on this problem using the Python-code in Listing 1 produces the results shown in Table 3.

Table 3 shows that the method, using the prescribed initial guess $d_0 = 7.00$ mm, converges on a diameter of $d_1^* \approx 19.13$ mm after three iterations. Increasing the initial guess in even increments reveals a *second* root at $d_2^* \approx 0.8314$ m, which is also evident from the plot of $f(d)$ in Figure 1. Since both these roots satisfy the heat transfer specifications we opt for the larger outer tube diameter — a larger tube diameter requires fewer tubes for the same transfer area A_{ex}, reducing both material cost and manufacturing complexity.

The observed order of convergence p varies across starting guesses but tend towards $p \approx 2$ as we approach a root, which is consistent with the theoretical *local* quadratic convergence of Newton's method. Cases where p deviates noticeably from the theoretical $p = 2$, such as at $d_0 = 353.5$ mm with $p \approx 0.917$, can be attributed to starting values that lie far from a root and therefore outside the local neighbourhood where the quadratic convergence theorem applies.[2] Inspecting Figure 1, we see that

[2] A known limitation of Newton's Method is that one has to select an initial value that is already sufficiently close to a root for the method to be numerically stable.

d_0 (mm)	d^* (mm)	Iterations	p	Scipy d^* (mm)	$\lvert d^* - \text{Scipy}\rvert$ (mm)
7.0000	19.131407149325682	3	2.332896	19.131407149308625	1.71×10^{-14}
64.7500	19.131407149308647	4	2.322647	19.131407149308629	2.08×10^{-17}
122.5000	19.131407149308622	5	2.086488	19.131407149308611	1.04×10^{-17}
180.2500	19.131407151041735	5	1.850623	19.131407149308608	1.73×10^{-12}
238.0000	19.131407149308654	6	1.548516	19.131407149308618	3.82×10^{-17}
295.7500	19.131407149308636	6	1.058250	19.131407149308622	1.39×10^{-17}
353.5000	831.392740792126460	5	0.917235	831.392740793674875	1.55×10^{-12}
411.2500	831.392740793626103	7	1.350690	831.392740793672374	4.63×10^{-14}
469.0000	831.392740793674875	7	1.635620	831.392740793674761	1.11×10^{-16}
526.7500	831.392740791717188	6	1.774351	831.392740793674648	1.96×10^{-12}
584.5000	831.392740793674875	6	1.856506	831.392740793666690	8.22×10^{-15}
642.2500	831.392740791158303	5	1.911815	831.392740793674761	2.52×10^{-12}
700.0000	831.392740793674761	5	1.951238	831.392740793674761	0.00×10^{0}

Table 3: Newton convergence results for all starting guesses d_0, compared against `scipy.optimize.fsolve`. The horizontal rule separates the two points of convergence: $d^* \approx 0.01913$ m and $d^* \approx 0.8314$ m.

the starting value $d_0 = 353.5$ mm lies in a region where $f(d)$ is very steep and very far from both of the two roots. In this region, the tangent-based update

$$d_{k+1} = d_k - \frac{f(d_k)}{f'(d_k)}$$

has step length

$$|d_{k+1} - d_k| = |-\frac{f(d_k)}{f'(d_k)}|$$

and can produce updates with large steps that move far across the graph due the still very large function values before the iterates starts trending towards a specific root. In this initial phase, the error sequence is dominated by the global shape of $f(d)$ rather than its local quadratic behavior near the root, so the observed order of convergence is closer to linear than quadratic. This can be observed in Figure 2 where the first step with starting guess $d_0 = 353.50$ mm appears notably more linear than subsequent steps.

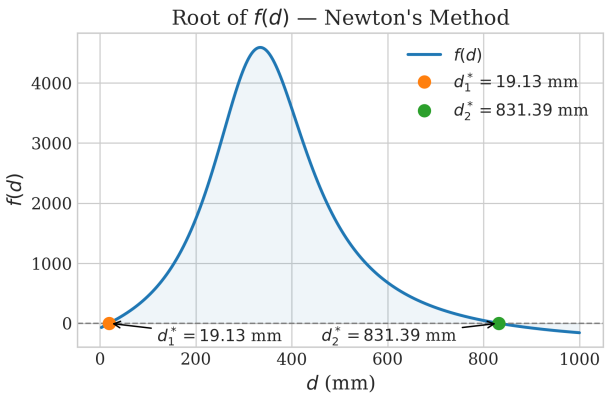


Figure 1: Graph of $f(d)$ over $d \in (0, 1]$ m, with the approximated roots marked. See Listing 1 for the implementation.

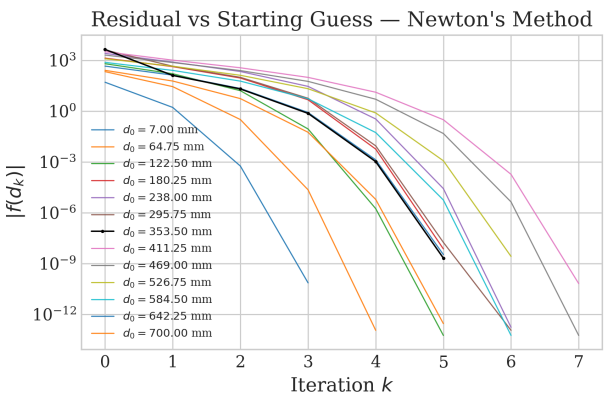


Figure 2: Residual $|d_k - d^*|$ versus iteration k for different starting guesses with Newton's method . See Listing 1 *"Order of Convergence"* section for implementation.

3.1.2 Numerical Solution of the non-linear system

Using the Python code in Listing 2 we simultaneously solve for both the outer tube diameter d and the inner shell diameter D_s by implementing Algorithm 2. This code produces the results shown in Table 4:

Table 4: Newton's Method for Nonlinear System (d, D_s)

k	d_k **(mm)**	$D_{s,k}$ **(mm)**	$\|F(x_k)\|_2$	$\|x_k - x^*\|_2$	p_k
0	15.000000	800.000000	5.58732501×10^{5}	$1.15566350 \times 10^{-1}$	–
1	14.512150	777.419100	2.41670069×10^{5}	$9.29805713 \times 10^{-2}$	0.262897
2	13.883940	772.267000	9.65862053×10^{4}	$8.78141296 \times 10^{-2}$	2.901072
3	13.145660	758.858800	3.38316968×10^{4}	$7.43938660 \times 10^{-2}$	2.157826
4	12.479150	736.485600	8.77327269×10^{3}	$5.20133509 \times 10^{-2}$	1.984354
5	12.085780	710.043100	1.28077228×10^{3}	$2.55683086 \times 10^{-2}$	1.531381
6	11.942580	693.093400	1.09162252×10^{2}	$8.61795814 \times 10^{-3}$	1.142165
7	11.902210	686.964200	9.17536055×10^{0}	$2.48864323 \times 10^{-3}$	1.023955
8	11.891160	685.173200	1.00172096×10^{0}	$6.97587461 \times 10^{-4}$	1.006337
9	11.888110	684.669600	$2.08936633 \times 10^{-1}$	$1.93969995 \times 10^{-4}$	1.001770
10	11.887270	684.529400	$5.62105801 \times 10^{-2}$	$5.38129237 \times 10^{-5}$	1.000495
11	11.887030	684.490500	$1.55480745 \times 10^{-2}$	$1.49197955 \times 10^{-5}$	1.000138
12	11.886970	684.479700	$4.30933832 \times 10^{-3}$	$4.13582602 \times 10^{-6}$	1.000039
13	11.886950	684.476700	$1.19450353 \times 10^{-3}$	$1.14640944 \times 10^{-6}$	1.000015
14	11.886950	684.475900	$3.31101187 \times 10^{-4}$	$3.17767263 \times 10^{-7}$	1.000017
15	11.886950	684.475700	$9.17765805 \times 10^{-5}$	$8.80783172 \times 10^{-8}$	1.000052
16	11.886940	684.475600	$2.54391336 \times 10^{-5}$	$2.44118045 \times 10^{-8}$	1.000185
17	11.886940	684.475600	$7.05135466 \times 10^{-6}$	$6.76437517 \times 10^{-9}$	1.000667
18	11.886940	684.475600	$1.95453191 \times 10^{-6}$	$1.87276678 \times 10^{-9}$	1.002413
19	11.886940	684.475600	$5.41767464 \times 10^{-7}$	$5.16884874 \times 10^{-10}$	1.008796
20	11.886940	684.475600	$1.50169740 \times 10^{-7}$	$1.41054272 \times 10^{-10}$	1.032962
21	11.886940	684.475600	$4.16249602 \times 10^{-8}$	$3.68797212 \times 10^{-11}$	1.138833
22	11.886940	684.475600	$1.15377952 \times 10^{-8}$	$8.00396433 \times 10^{-12}$	–
23	11.886940	684.475600	$3.19796218 \times 10^{-9}$	0.00000000×10^{0}	–

The method converges on values $d = 11.886940$ mm and $D_s = 684.475600$ mm for the sought after diameters. However, in the results we note that the order of convergence is not $p \approx 2$ as expected. Studying Figure 3 and Figure 4 we conclude that the convergence appear linear instead of the theoretically expected quadratic behaviour. This is most likely due to the Jacobian being ill conditioned and close to singular near the solution.

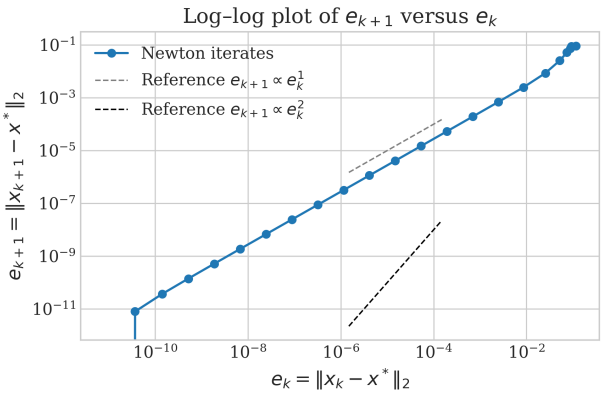


Figure 3: See Listing 2 for the implementation.

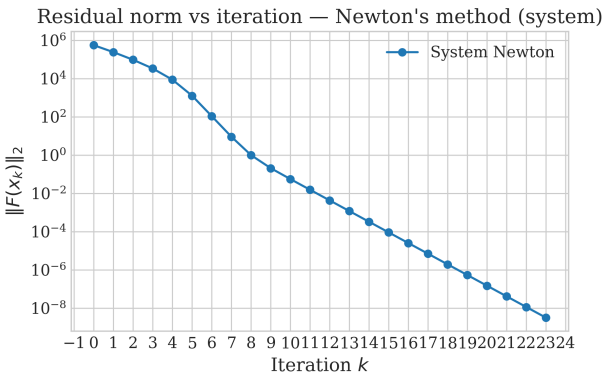


Figure 4: Residual $|x_k - x^*|$ versus iteration k for Newton's method. See Listing 2 *"Order of Convergence"* section for implementation.

Part 2 - Heat Sinks

3.1.3 Verification for the infinite length case

To get some sense of whether or not our implementation seems reasonable, we test it against a case where the heat sink is so long that at some sufficiently large x we expect the rod to reach ambient temperature T_∞ and where radiative effects are neglected, this case has the analytical solution:

$$T(x) = T_\infty + (T_s - T_\infty)e^{-\sqrt{\alpha_1}x}. \tag{16}$$

We use the values in Table 5 and approximate the temperature distribution function $T(x)$ for $N = 400$ using the code in Listing 3 and compare the results with Equation (16), the results of which is shown in Figure 6 and Figure 5.

Table 5: Heat Sink parameter values for implementation test.

Parameter	**Value**	**Unit**
h_c	40	$\frac{W}{m^2 \cdot K}$
K	240	$\frac{W}{m \cdot K}$
ϵ	0.4	
σ	$5.67 \cdot 10^{-8}$	$\frac{W}{m^2 K^4}$
D	$4.13 \cdot 10^{-3}$	m
T_s	450	K (Kelvin)
T_∞	293	K (Kelvin)

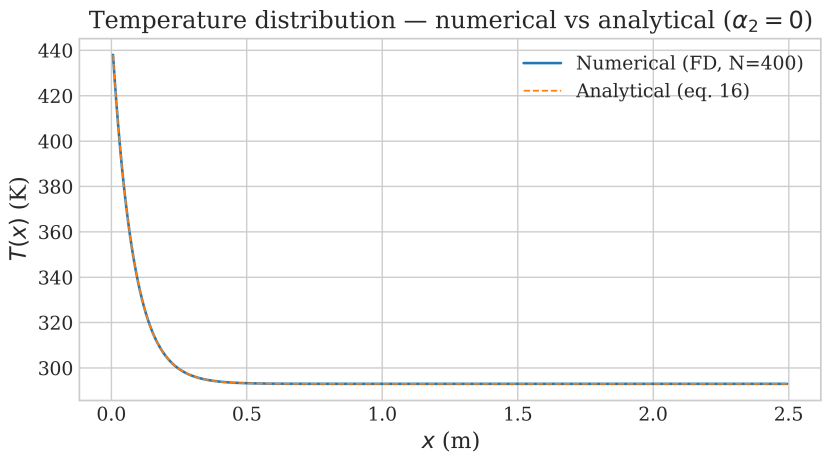


Figure 5: Temperature distribution of our approximate Newton solution compared to the analytical case Equation (16)

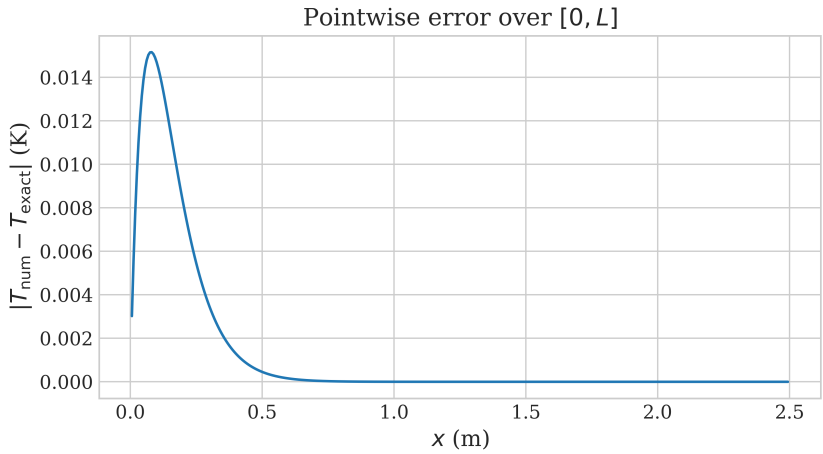


Figure 6: Pointwise error of our approximate Newton solution compared to the analytical case Equation (16).

Judging from Figure 5, the numerical approximation follows the analytical solution closely enough for practical use. For small values of x, however, Figure 6 shows a relatively large bump in the pointwise error. This is a boundary effect: the fixed Dirichlet condition at $x = 0$ makes the first interior node strongly depend on the prescribed value, so the leading $O(h^2)$ error term of a Taylor expansion around this point might still be quite large. As a result, the first "step" away from the boundary is less accurate, even though the method remains globally $O(h^2)$ accurate.

3.1.4 Method order of accuracy

Using the same coefficients as in the previous section, we now evaluate the error as a function of N. For each $N \in \{50, 100, 200, 400, 800\}$ we solve the boundary value problem numerically and compare the result to the analytical reference solution Equation (16). The discrete 2-norm of the error is defined as

$$\|\mathbf{r}\|_2 = \left(\frac{1}{N-1} \sum_{k=1}^{N-1} |r_k|^2 \right)^{1/2}, \tag{17}$$

where $r_k = T_N(x_k) - T_{\text{ref}}(x_k)$ is the pointwise error at the interior grid point x_k.
To estimate the observed order of accuracy p we use consecutive pairs (N_{k-1}, N_k) and their corresponding

errors $e_{N_{k-1}}, e_{N_k}$:

$$p_k = \frac{\log\left(\dfrac{e_{N_{k-1}}}{e_{N_k}}\right)}{\log\left(\dfrac{N_k}{N_{k-1}}\right)}. \tag{18}$$

If the method is second-order, we expect $p_k \approx 2$ for sufficiently large N.
The results are presented in Table 6 and Figure 7, the discrete 2-norm e_N decreases approximately like N^{-2}, and the estimated orders p_k are close to 2, which confirms that the finite difference method is globally second-order accurate.

N	e_N	p
50	2.3051×10^{-1}	—
100	5.8648×10^{-2}	1.9747
200	1.4681×10^{-2}	1.9981
400	3.6668×10^{-3}	2.0014
800	9.1589×10^{-4}	2.0013

Table 6: Discrete 2-norm error e_N and estimated order of accuracy p for different number of intervals N.

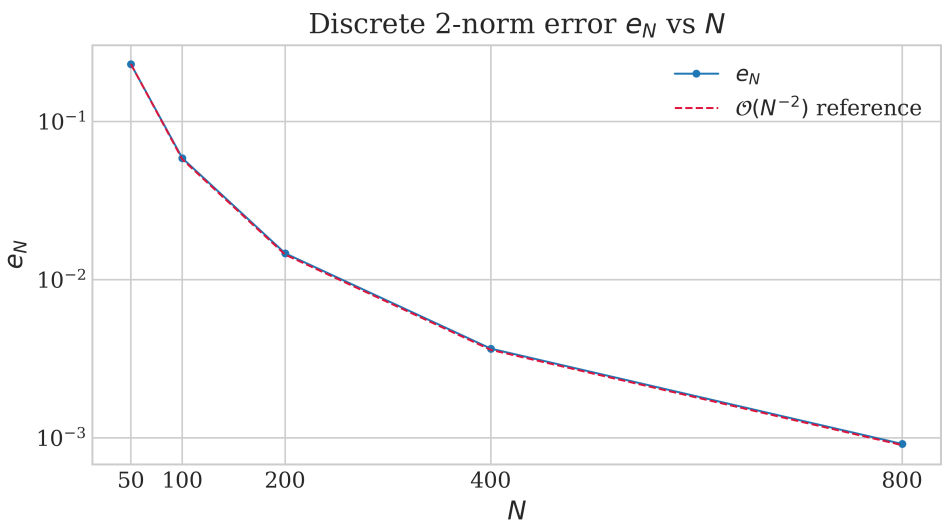


Figure 7: Discrete 2-norm error as a function of N.

3.1.5 Non-Linear Case: SS AISI 316, Aluminium, and Copper

We now include the second term ($\alpha_2 \neq 0$) and solve the full non-linear Equation (15) for three materials: stainless steel (AISI 316), aluminium alloy, and copper.
The geometry and boundary conditions are equal for all materials and is: $L = 0.30$ m, $D = 5.0 \times 10^{-3}$ m, $T(0) = 373.15$ K, and $T(L) = T_\infty = 293.15$ K. The material-specific coefficients are

$$\alpha_1 = \frac{4h_c}{DK}, \qquad \alpha_2 = \frac{4\epsilon\sigma}{DK}, \tag{19}$$

with σ being Boltzmann's constants[3] and the remaining parameters taken from Table 7.

[3] $\sigma = 5.67 \times 10^{-8}\ \mathrm{W\,m^{-2}\,K^{-4}}$

Material	K $[\frac{\mathrm{W}}{\mathrm{m\,K}}]$	h_c $[\frac{\mathrm{W}}{\mathrm{m^2K}}]$	ϵ	α_1 $[\mathrm{m}^{-2}]$	α_2 $[\mathrm{m}^{-2}\mathrm{K}^{-3}]$
SS AISI 316	14	100	0.17	5714.29	5.508×10^{-7}
Aluminium	180	100	0.82	444.44	2.066×10^{-7}
Copper	398	100	0.03	201.00	3.419×10^{-9}

Table 7: Material parameters and computed coefficients.

Using the code in Listing 3 with $N = 400$, the temperature distributions for each material are plotted in Figure 8.

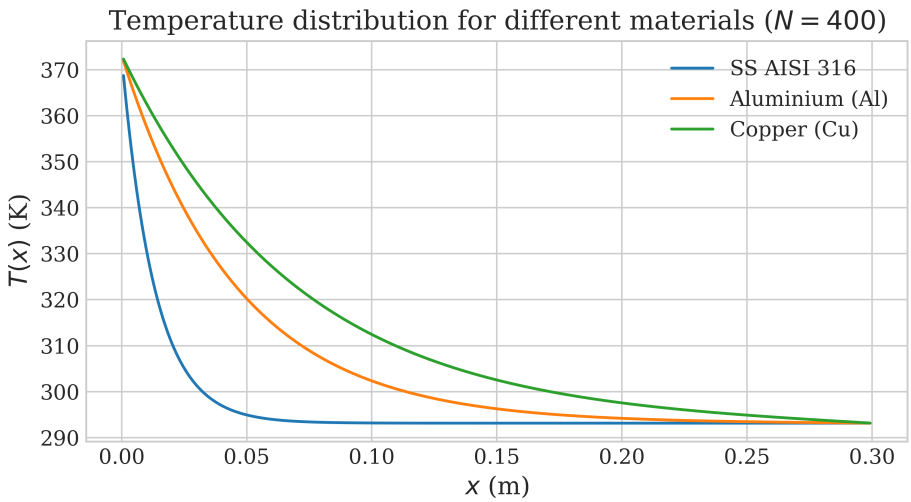


Figure 8: Temperature distribution $T(x)$ along the heat sink for three materials, with ($\alpha_2 \neq 0$), $N = 400$.

The three profiles in Figure 8 differ substantially, why that is is evident directly from α_1 in Table 7. Since $\alpha_1 \propto K^{-1}$, a lower thermal conductivity produces a *larger* α_1 and therefore a steeper exponential decay toward T_∞. The stainless steel drop to near-ambient temperature within the first few centimeters due to it's poor thermal conductivity. The aluminum decays more gradually, while the copper disperses the energy the most, maintaining an elevated temperature almost across the full 0.30 m length. α_2 plays a minor role here since its values are several orders of magnitude smaller than α_1.

Required length for the infinite assumption. The analytical solution for the infinite case (Equation (16)) assumes that the temperature decreases exponentially and to an infinitesimal degree approaches the ambient temperature as $x \to \infty$. If we apply a threshold for when we may say that this approximation is valid, e.g when the deviation is less than 1%, we get a practical first estimate of a cut-off length:

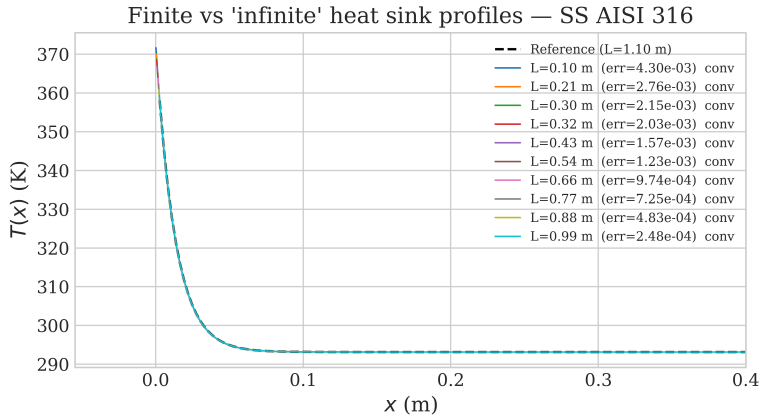
$$e^{-\sqrt{\alpha_1}\,L} < 0.01 \iff -\sqrt{\alpha_1}\,L < \ln(0.01) \iff L > \frac{-\ln(0.01)}{\sqrt{\alpha_1}} \approx \frac{4.6}{\sqrt{\alpha_1}}. \tag{20}$$

Applying Equation (20) to each material gives the minimum lengths in Table 8.

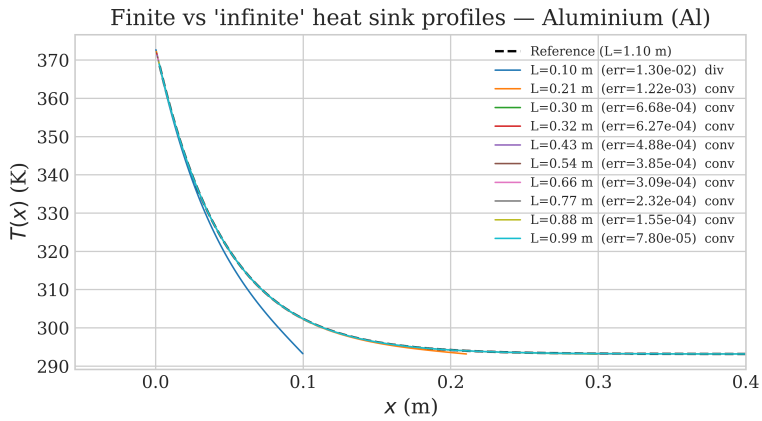
Material	$\sqrt{\alpha_1}$ $[\mathrm{m}^{-1}]$	$L_{\min} = 4.6/\sqrt{\alpha_1}$ [m]	Valid at $L = 0.30$ m?
SS AISI 316	75.6	0.061	Yes
Aluminium	21.1	0.218	Yes
Copper	14.2	0.324	**No**

Table 8: Minimum length for the infinite-length assumption to hold ($< 1\%$ error at the tip).

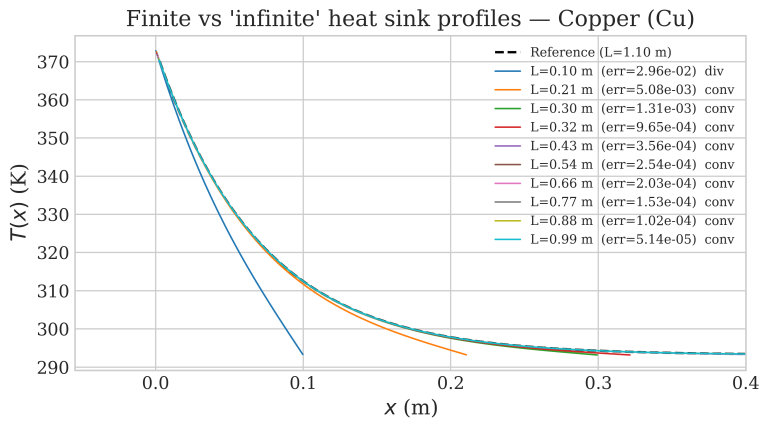
The stainless steel satisfies the criterion at just 6.1 cm since its low conductivity traps the majority of energy near the base. The aluminium requires roughly 22 cm, which is within the simulated 30 cm. And lastly, the copper *does not* satisfy the criterion at $L = 0.30$ m: its high conductivity means the temperature disperses rapidly and well throughout and has not decayed sufficiently by the tip. Therefore the infinite-length formula Equation (16) would underestimate the tip temperature and overestimate the total heat loss for copper heat sinks of this length.
Using Listing 3 we calculate the full temperature distribution for several lengths for each of the different materials using Newton's Method as described in Section 2.2.2, the results are presented in Figure 9a, Figure 9b, Figure 9c:



(a) SS AISI 316



(b) Aluminium



(c) Copper

Figure 9: Temperature distribution $T(x)$ for different materials.

4 Discussion

References

A Python Source Code

Heat Exchanger – Newton's Method for non-linear equation

Listing 1: Heat Exchanger – Newton's Method for non-linear equation

```python
"""
SF1546 - Project VT2026
24/3/2026
Group 41

"""

from pathlib import Path

import numpy as np
import scipy.optimize as opt
from utils import NewtonRaphson, NewtonRaphsonResult

ROOT = Path(__file__).parent
plot_dir: Path = ROOT / "Plots"
plot_dir.mkdir(exist_ok=True)
# Results Save directory
results_path: Path = ROOT / "Results"
results_path.mkdir(exist_ok=True)

def main():
    # ============= Setup =============
    Q: float = 801368  # Heat Flux
    Aex: float = 64.15  # Heat transfer area
    dTm: float = 29.6  # Mean temp. delta of the two fluids
    Ds: float = 1.219  # Shell inner diameter
    # Material and Thermodynamic parameters:
    Rfi: float = 1.76e-4
    Rfo: float = 1.76e-4
    hs: float = 356
    ht: float = 356
    kw: float = 60

    U_f: float = Q / (Aex * dTm)  # Heat transfer coefficient
    d_0: float = 0.007  # Reference initial guess
    tolerance: float = 1e-8

    def g(d) -> float:
        return (
            d / (Ds * ht) + d * Rfi / Ds + d * np.log(d / Ds) / (2 * kw) + Rfo + 1 / hs
        )

    def g_prime(d) -> float:
        return 1 / (Ds * ht) + Rfi / Ds + (np.log(d / Ds) + 1) / (2 * kw)

    def f(d) -> float:
        return 1 / g(d) - U_f

    def f_prime(d) -> float:
        return -g_prime(d) / (g(d) ** 2)

    starting_vals: np.ndarray = np.linspace(d_0, 0.7, 13)

    # Run the reference Newton (for summary)
```

```python
    ref_result: NewtonRaphsonResult = NewtonRaphson(
        f=f, f_prime=f_prime, x0=d_0, tolerance=tolerance, max_iterations=100
    )
    ref_root = ref_result.final_x
    ref_residual = abs(f(ref_root))

    # ============= Results file =============
    results_file = results_path / "Newton Scalar Results.txt"
    with results_file.open("w", encoding="utf-8") as out:
        # Header to match system style
        out.write("Newton's Method for Scalar Nonlinear Equation (d)\n")
        out.write("=" * 100 + "\n\n")

        # Summary for reference run
        out.write("Reference run (starting from d0 = 0.007 m):\n")
        out.write(f"  d*   = {ref_root: .8e} m\n")
        out.write(f"  |f(d*)| = {ref_residual: .3e}\n")
        out.write(f"  Iterations = {len(ref_result.xs) - 1:d}\n\n")

        # Table header for all starting guesses (similar style to before)
        out.write(
            f"{'d0 [mm]':>12}  {'d* [mm]':>22}  {'Iterations':>10}  "
            f"{'p (first)':>10}  {'Scipy d* [mm]':>22}  {'|d* - Scipy|':>15}\n"
        )
        out.write("-" * 100 + "\n")

        # ============= Loop over starting guesses =============
        for x0 in starting_vals:
            results: NewtonRaphsonResult = NewtonRaphson(
                f=f, f_prime=f_prime, x0=x0, tolerance=tolerance, max_iterations=100
            )

            # Order of convergence (estimate from error to final_x)
            errors: list[float] = [abs(x - results.final_x) for x in results.xs]
            p: list[float] = [
                np.log(abs(errors[k + 1] / errors[k]))
                / np.log(abs(errors[k] / errors[k - 1]))
                for k in range(1, len(errors) - 1)
                if errors[k] > 0 and errors[k - 1] > 0 and errors[k + 1] > 0
            ]

            # Verification with SciPy
            root, info, exitflag, output = opt.fsolve(
                func=f, x0=x0, fprime=f_prime, full_output=True
            )
            scipy_result = root[0]

            # Write one row for this starting guess
            p_str = f"{p[0]:.6f}" if p else "N/A"
            delta_scipy = abs(results.final_x - scipy_result)
            out.write(
                f"{x0 * 1000:>11.4f}  "  # x0 in mm
                f"{results.final_x * 1000:>22.15f}  "  # d* in mm
                f"{len(results.xs) - 1:>10}  "
                f"{p_str:>10}  "
                f"{scipy_result * 1000:>22.15f}  "  # scipy d* in mm
                f"{delta_scipy:>15.2e}\n"
```

```
            )

    print(f"Scalar Newton results written to {results_file}")

if __name__ == "__main__":
    main()
```

Heat Exchanger – Newton's Method for non-linear system

Listing 2: Heat Exchanger – Newton's Method for non-linear system

```
"""
SF1546 - Project VT2026
24/3/2026
Group 41
"""

from pathlib import Path

import numpy as np
from utils.methods import newton_system

# Plot save directory
ROOT = Path(__file__).parent
plot_dir: Path = ROOT / "Plots"
plot_dir.mkdir(exist_ok=True)
# Results Save directory
results_path: Path = ROOT / "Results"
results_path.mkdir(exist_ok=True)

# ============= Model Definition =============

def g(
    d: float, Ds: float, ht: float, Rfi: float, kw: float, Rfo: float, hs: float
) -> float:
    """
    g(d, Ds) from the report:
        g(d, D_s) = d/(D_s h_t) + d R_fi / D_s + d ln(d/D_s)/(2 k_w) + R_fo + 1/h_s
    """
    return (
        d / (Ds * ht) + d * Rfi / Ds + d * np.log(d / Ds) / (2.0 * kw) + Rfo + 1.0 / hs
    )

def g_d(d: float, Ds: float, ht: float, Rfi: float, kw: float) -> float:
    """
    dg/dd from the report:
        g_d = 1/(D_s h_t) + R_fi / D_s + (ln(d/D_s) + 1)/(2 k_w)
    """
    return 1.0 / (Ds * ht) + Rfi / Ds + (np.log(d / Ds) + 1.0) / (2.0 * kw)

def g_Ds(d: float, Ds: float, ht: float, Rfi: float, kw: float) -> float:
    """
    dg/dD_s from the report:
        g_{D_s} = -d / (D_s^2 h_t) - d R_fi / D_s^2 - d/(2 k_w D_s)
    """
    return -d / (Ds**2 * ht) - d * Rfi / (Ds**2) - d / (2.0 * kw * Ds)

```

```python
# Newton Function vector
def F(
    x: np.ndarray,
    Q: float,
    dTm: float,
    K1: float,
    n1: float,
    c: float,
    S_t: float,
    hs: float,
    ht: float,
    Rfi: float,
    Rfo: float,
    kw: float,
    dP_max: float,
) -> np.ndarray:
    """
    Vector function F(d, Ds) = (F1, F2)^T corresponding to eqs. (F1) and (F2) in eq. (11) in report.

    F1(d, Ds) = g(d, Ds)^(-1) - Q d^{n1-1} / (pi K1 Ds^{n1} DeltaT_m)
    F2(d, Ds) = c / (Ds^2 (S_t - d)^2) - DeltaP_max
    """
    d, Ds = x
    g_val = g(d, Ds, ht, Rfi, kw, Rfo, hs)

    F1 = (1.0 / g_val) - Q * d ** (n1 - 1.0) / (np.pi * K1 * Ds**n1 * dTm)
    F2 = c / (Ds**2 * (S_t - d) ** 2) - dP_max

    return np.array([F1, F2], dtype=float)

# Jacobian for Newton Function vector
def JF(
    x: np.ndarray,
    Q: float,
    dTm: float,
    K1: float,
    n1: float,
    c: float,
    S_t: float,
    hs: float,
    ht: float,
    Rfi: float,
    Rfo: float,
    kw: float,
    dP_max: float,
) -> np.ndarray:
    """
    Jacobian matrix JF(d, Ds) corresponding to eq. (12) in report:

        JF = [[dF1/dd,  dF1/dDs],
              [dF2/dd,  dF2/dDs]]
    """
    d, Ds = x
    g_val = g(d, Ds, ht, Rfi, kw, Rfo, hs)
    gd = g_d(d, Ds, ht, Rfi, kw)
    gDs = g_Ds(d, Ds, ht, Rfi, kw)

    # dF1/dd
    J11 = -gd / (g_val**2) + (n1 - 1.0) * Q * (d ** (n1 - 2.0)) / (
        np.pi * K1 * Ds**n1 * dTm
```

```
    )

    # dF1/dDs
    J12 = gDs / (g_val**2) + n1 * Q * d ** (n1 - 1.0) / (
        np.pi * K1 * Ds ** (n1 + 1.0) * dTm
    )

    # dF2/dd
    J21 = 2.0 * c / (Ds**2 * (S_t - d) ** 3)

    # dF2/dDs
    J22 = -2.0 * c / (Ds**3 * (S_t - d) ** 2)

    return np.array([[J11, J12], [J21, J22]], dtype=float)

# ============= Main  =============

def main() -> None:
    # ------------- Parameters -------------
    # Heat / thermo parameters
    Q: float = 801_368.0  # Heat flux
    dTm: float = 29.6  # Mean temp. difference
    dP_max: float = 49_080.0  # Max allowed pressure drop

    # Material parameters
    c: float = 0.389
    S_t: float = 0.016
    K_1: float = 0.249
    n_1: float = 2.207

    # Thermal parameters / coefficients
    Rfi: float = 1.76e-4
    Rfo: float = 1.76e-4
    hs: float = 356.0
    ht: float = 356.0
    kw: float = 60.0

    # Newton settings
    tol: float = 1e-8
    max_iter: int = 50

    # Initial guess for (d, Ds)
    d0: float = 0.015
    Ds0: float = 0.8
    x0 = np.array([d0, Ds0], dtype=float)

    # Pack parameters in the order F, JF expect
    params = (Q, dTm, K_1, n_1, c, S_t, hs, ht, Rfi, Rfo, kw, dP_max)

    # ============= Newton solve =============
    result = newton_system(
        F_func=F,
        JF_func=JF,
        x0=x0,
        tol=tol,
        max_iter=max_iter,
        args=params,
    )

    d_star, Ds_star = result.final_x
    residual_norm = np.linalg.norm(result.Fs[-1], ord=2)
```

```python

    print(f"Converged in {len(result.ns) - 1} iterations")
    print(f"d*  = {d_star:.6e} m")
    print(f"Ds* = {Ds_star:.6e} m")
    print(f"||F(x*)||_2 = {residual_norm:.3e}")

    # ============= Convergence analysis =============

    # ------------- Per-iteration residuals and errors -------------
    residuals = [np.linalg.norm(Fk, ord=2) for Fk in result.Fs]
    x_star = result.final_x
    errors = [np.linalg.norm(x_k - x_star, ord=2) for x_k in result.xs]

    # Pairs of e_k, e_k+1 for error plot
    e_k = np.array(errors[:-1])
    e_kp1 = np.array(errors[1:])

    # ------------- Per-iteration Jacobian determinant and condition number -------------
    det_Js: list[float] = []
    cond_Js: list[float] = []
    for x_k in result.xs:
        J_k = JF(x_k, *params)
        det_Js.append(np.linalg.det(J_k))
        cond_Js.append(np.linalg.cond(J_k))

    # ------------- Per-iteration order p_k (aligned with k) -------------
    p_vals: list[float | None] = [None] * len(errors)
    for k in range(1, len(errors) - 1):
        e_km1, e_k, e_kp1 = errors[k - 1], errors[k], errors[k + 1]
        if e_km1 > 0 and e_k > 0 and e_kp1 > 0:
            p_k = np.log(e_kp1 / e_k) / np.log(e_k / e_km1)
            p_vals[k] = p_k

    # Jacobian at final iterate (for summary)
    J_star = JF(x_star, *params)
    det_J = np.linalg.det(J_star)
    cond_J = np.linalg.cond(J_star)

    # ============= Write Results to .txt file =============
    results_file = results_path / "Newton System Results.txt"
    with results_file.open("w", encoding="utf-8") as out:
        out.write("Newton's Method for Nonlinear System (d, D_s)\n")
        out.write("=" * 80 + "\n\n")
        out.write("Initial Guess:\n")
        out.write(f"d_0 = {d0}\n")
        out.write(f"D_s0 = {Ds0}\n\n")
        out.write("Final solution:\n")
        out.write(f"  d*   = {d_star: .8e} m\n")
        out.write(f"  D_s* = {Ds_star: .8e} m\n")
        out.write(f"  ||F(x*)||_2 = {residual_norm: .3e}\n")
        out.write(f"  Iterations  = {len(result.ns) - 1:d}\n\n")

        out.write("Jacobian at x*:\n")
        out.write(f"  det(J(x*))  = {det_J: .8e}\n")
        out.write(f"  cond(J(x*)) = {cond_J: .3e}\n\n")

        out.write("Per-iteration data:\n")
        out.write(
            f"{'k':>4}  {'d_k [mm]':>12}  {'D_s,k [mm]':>12}  "
            f"{'||F(x_k)||_2':>18}  {'||x_k - x*||_2':>18}  "
```

```
            f"{'det(J_k)':>12}  {'cond(J_k)':>12}  {'p_k':>12}\n"
        )
        out.write("-" * 120 + "\n")

        for k, (x_k, res_k, err_k, det_k, cond_k, p_k) in enumerate(
            zip(result.xs, residuals, errors, det_Js, cond_Js, p_vals)
        ):
            d_k, Ds_k = x_k
            if p_k is not None:
                out.write(
                    f"{k:4d}  {d_k * 1000:12.6e}  {Ds_k * 1000:12.6e}  "
                    f"{res_k:18.8e}  {err_k:18.8e}  "
                    f"{det_k:12.4e}  {cond_k:12.3e}  {p_k:12.6f}\n"
                )
            else:
                out.write(
                    f"{k:4d}  {d_k * 1000:12.6e}  {Ds_k * 1000:12.6e}  "
                    f"{res_k:18.8e}  {err_k:18.8e}  "
                    f"{det_k:12.4e}  {cond_k:12.3e}  {'-':>12}\n"
                )

    print(f"Results written to {results_file}")

if __name__ == "__main__":
    main()
```

Heat Sink – Newton's Method for non-linear system

Listing 3: Heat Sink – Newton's Method for non-linear system

```
from pathlib import Path

import numpy as np
from utils.methods import newton_system

# ============= Paths =============
ROOT = Path(__file__).parent
plot_dir: Path = ROOT / "Plots"
plot_dir.mkdir(exist_ok=True)

# ============= Model Definition =============

def make_A(N: int) -> np.ndarray:
    """Tridiagonal (N-1) x (N-1) FD matrix with -2 on diagonal, +1 on off-diagonals."""
    n = N - 1
    return (
        np.diag([-2] * n) + np.diag([1] * (n - 1), k=1) + np.diag([1] * (n - 1), k=-1)
    )

def f(T: float, T_inf: float) -> float:
    return T**4 - T_inf**4

def f_prime(T: float) -> float:
    return 4 * T**3

def interior_grid(N: int, L: float) -> np.ndarray:
    """Interior grid points x_1, ..., x_{N-1}."""
    h = L / N
```

```python
    return np.linspace(h, L - h, N - 1)

def rhs_vector(
    a: float,
    b: float,
    h: float,
    T_s: float,
    T_L: float,
    T: np.ndarray,
    T_inf: float,
    f: callable,
) -> np.ndarray:
    """RHS vector r(T), same shape as T."""
    rhs = h**2 * (a * (T - T_inf) + b * np.vectorize(f)(T, T_inf))
    rhs[0] -= T_s
    rhs[-1] -= T_L
    return rhs

def F_func(
    T: np.ndarray,
    A: np.ndarray,
    a: float,
    b: float,
    h: float,
    T_s: float,
    T_L: float,
    T_inf: float,
    f: callable,
) -> np.ndarray:
    return A @ T - rhs_vector(a, b, h, T_s, T_L, T, T_inf, f)

def JF_func(
    T: np.ndarray,
    A: np.ndarray,
    a: float,
    b: float,
    h: float,
    f_prime: callable,
) -> np.ndarray:
    d = -(h**2) * (a + b * np.vectorize(f_prime)(T))
    return A + np.diag(d)

def discrete_2_norm(r_vec: np.ndarray) -> float:
    """Discrete 2-norm (eq. 16): sqrt(1/(N-1) * sum |r_k|^2)."""
    return np.sqrt(np.sum(r_vec**2) / (len(r_vec) - 1))

def solve(
    N: int,
    L: float,
    a: float,
    b: float,
    T_s: float,
    T_L: float,
    T_inf: float,
) -> tuple:
    """Run Newton solver and return (result, x_grid)."""
    h = L / N
    A_mat = make_A(N)
    T0 = np.linspace(T_s, T_L, N - 1)
    result = newton_system(
        F_func,
```

```python
        JF_func,
        T0,
        tol=1e-10,
        max_iter=1000,
        args=(A_mat, a, b, h, T_s, T_L, T_inf, f),
        jac_args=(A_mat, a, b, h, f_prime),
    )
    return result, interior_grid(N, L)

# ============= 8.3 =============

def run_83() -> None:
    # ------------- Parameters (Table 3) -------------
    h_c: float = 40
    K: float = 240
    D: float = 4.13e-3
    T_s: float = 450
    T_inf: float = 293
    T_L: float = T_inf
    L: float = 2.5

    a: float = (4 * h_c) / (D * K)  # alpha_1
    b: float = 0.0  # alpha_2 = 0

    Ns: list[int] = [50 * 2**k for k in range(5)]  # [50, 100, 200, 400, 800]
    N_main: int = 400

    results = {}
    grids = {}
    for N in Ns:
        results[N], grids[N] = solve(N, L, a, b, T_s, T_L, T_inf)

    # ------------- 8.3.a: N=400 vs analytical -------------
    T_num = results[N_main].final_x
    x_grid = grids[N_main]
    T_exact = T_inf + (T_s - T_inf) * np.exp(-np.sqrt(a) * x_grid)
    r_vec = T_num - T_exact
    e_400 = discrete_2_norm(r_vec)

    print("=== 8.3.a ===")
    print(f"Converged in {len(results[N_main].ns) - 1} iterations")
    print(f"Discrete 2-norm ||r||_2 = {e_400:.3e} K")
    print(f"Max pointwise error:      {np.abs(r_vec).max():.3e} K")
    print(f"Error at x=h:             {np.abs(r_vec[0]):.3e} K")
    print(f"Error at x=L-h:           {np.abs(r_vec[-1]):.3e} K")

    fig1, ax1 = plt.subplots(figsize=(7, 4))
    ax1.plot(x_grid, T_num, lw=1.5, label=f"Numerical (FD, N={N_main})")
    ax1.plot(x_grid, T_exact, lw=1.0, linestyle="--", label="Analytical (eq. 17)")
    ax1.set_xlabel(r"$x$ (m)")
    ax1.set_ylabel(r"$T(x)$ (K)")
    ax1.set_title(
        r"Temperature distribution - numerical vs analytical ($\alpha_2 = 0$)"
    )
    ax1.legend(frameon=False)
    fig1.tight_layout()
    fig1.savefig(plot_dir / "8_3a_solution.png", bbox_inches="tight")

```

```python
    fig2, ax2 = plt.subplots(figsize=(7, 4))
    ax2.plot(x_grid, np.abs(r_vec), lw=1.5)
    ax2.set_xlabel(r"$x$ (m)")
    ax2.set_ylabel(r"$|T_{\mathrm{num}} - T_{\mathrm{exact}}|$ (K)")
    ax2.set_title(r"Pointwise error over $[0, L]$")
    fig2.tight_layout()
    fig2.savefig(plot_dir / "8_3a_error.png", bbox_inches="tight")

    # ------------- 8.3.b: Convergence study -------------
    print("\n=== 8.3.b ===")
    norms = []
    for N in Ns:
        T_num_N = results[N].final_x
        T_exact_N = T_inf + (T_s - T_inf) * np.exp(-np.sqrt(a) * grids[N])
        norms.append(discrete_2_norm(T_num_N - T_exact_N))

    # Order of accuracy: eN = Ch^p = C(L/N)^p -> p = log(e_k/e_{k+1}) / log(N_{k+1}/N_k)
    orders = [None] + [
        np.log(norms[k - 1] / norms[k]) / np.log(Ns[k] / Ns[k - 1])
        for k in range(1, len(Ns))
    ]

    print(f"\n{'N':>6}  {'e_N':>12}  {'order p':>10}")
    print("-" * 34)
    for N, e, p in zip(Ns, norms, orders):
        p_str = f"{p:.4f}" if p is not None else "-"
        print(f"{N:>6}  {e:>12.4e}  {p_str:>10}")

    fig3, ax3 = plt.subplots(figsize=(7, 4))
    ax3.semilogy(Ns, norms, "o-", lw=1.0, ms=3, label=r"$e_N$")
    ref = norms[0] * (Ns[0] / np.array(Ns)) ** 2
    ax3.semilogy(
        Ns, ref, "--", lw=1, color="crimson", label=r"$\mathcal{O}(N^{-2})$ reference"
    )
    ax3.set_xlabel(r"$N$")
    ax3.set_ylabel(r"$e_N$")
    ax3.set_title(r"Discrete 2-norm error $e_N$ vs $N$")
    ax3.set_xticks(Ns)
    ax3.legend(frameon=False)
    fig3.tight_layout()
    fig3.savefig(plot_dir / "8_3b_convergence.png", bbox_inches="tight")

# ============= 8.4 =============

def run_84() -> None:
    # ------------- Parameters (8.4) -------------
    sigma: float = 5.67e-8
    D: float = 5.0e-3
    T_s: float = 373.15
    T_inf: float = 293.15
    T_L: float = T_inf
    L_84a: float = 0.30  # length for 8.4.a
    N: int = 400

    # ------------- Table 4 -------------
    materials = ["SS AISI 316", "Aluminium (Al)", "Copper (Cu)"]
    Ks: list[float] = [14, 180, 398]
    h_cs: list[float] = [100, 100, 100]
    epss: list[float] = [0.17, 0.82, 0.03]
```

```python

    # ============= 8.4 Calculations =============
    L_tests: list[float] = [L_84a] + [0.1 + k for k in np.linspace(0, 1, 10)
        ]

    results_84: dict[str, dict[float, np.ndarray]] = {}
    grids_84: dict[str, dict[float, np.ndarray]] = {}

    for mat, K_i, h_c_i, eps_i in zip(materials, Ks, h_cs, epss):
        results_84[mat] = {}
        grids_84[mat] = {}

        a_i = (4 * h_c_i) / (D * K_i)
        b_i = (4 * eps_i * sigma) / (D * K_i)

        for L in L_tests:
            result, x_grid = solve(N, L, a_i, b_i, T_s, T_L, T_inf)
            results_84[mat][L] = result
            grids_84[mat][L] = x_grid

    # ------------- 8.4.a Results -------------
    print("\n=== 8.4.a ===")

    L_first = L_tests[0]

    fig4, ax4 = plt.subplots(figsize=(7, 4))

    for mat in materials:
        result = results_84[mat][L_first]
        x_grid = grids_84[mat][L_first]

        print(f"{mat}: converged in {len(result.ns) - 1} iterations")

        ax4.plot(x_grid, result.final_x, lw=1.5, label=mat)

    ax4.set_xlabel(r"$x$ (m)")
    ax4.set_ylabel(r"$T(x)$ (K)")
    ax4.set_title(r"Temperature distribution for different materials ($N=400
        $)")
    ax4.legend(frameon=False)
    fig4.tight_layout()
    fig4.savefig(plot_dir / "8_4a_materials.png", bbox_inches="tight")

    # ------------- 8.4.b Results -------------
    tol_frac = 0.01  # e.g. 1% relative error

    print("\n=== 8.4.b ===")

    for mat in materials:
        # Sort lengths to have a clear order
        Ls_sorted = sorted(L_tests)
        L_ref = Ls_sorted[-1]  # longest length as 'infinite' reference

        # Reference solution for this material
        result_ref = results_84[mat][L_ref]
        x_ref = grids_84[mat][L_ref]
        T_ref = result_ref.final_x

        # Compute relative profile error for each shorter length
        rel_errs = {}
        for L in Ls_sorted[:-1]:  # exclude L_ref
```

```python
            result_L = results_84[mat][L]
            x_L = grids_84[mat][L]
            T_L = result_L.final_x

            # Interpolate reference onto x_L
            T_ref_on_L = np.interp(x_L, x_ref, T_ref)

            diff = T_L - T_ref_on_L
            err = discrete_2_norm(diff)
            ref_norm = discrete_2_norm(T_ref_on_L)
            rel_err = err / ref_norm if ref_norm > 0 else 0.0

            rel_errs[L] = rel_err

        # Pick smallest L with rel_err < tol_frac
        L_min = next((L for L in Ls_sorted[:-1] if rel_errs[L] < tol_frac), None)

        print(f"\n{mat}: reference length L_ref = {L_ref:.2f} m")
        print(f"  {'L (m)':>8}  {'rel_err':>12}  {'converged':>10}")
        print("  " + "-" * 34)
        for L in Ls_sorted[:-1]:
            re = rel_errs[L]
            print(f"  {L:>8.2f}  {re:>12.4e}  {'yes' if re < tol_frac else 'no':>10}")
        print(rf"  -> Estimated L_min \approx {L_min} m")

# ============= Entry Point =============

def main() -> None:
    run_83()
    run_84()

if __name__ == "__main__":
    main()
```

Structures

Listing 4: Structures

```python
from dataclasses import dataclass

import numpy as np

@dataclass
class NewtonRaphsonResult:
    """
    Result container for scalar Newton-Raphson iterations.

    Parameters
    ------
    fs : list[float]
        Sequence of function values f(x_k) at each iteration k.
    xs : list[float]
        Sequence of iterates x_k produced by Newton's method.
    ns : list[int]
        Iteration indices corresponding to each x_k (typically [0, 1, ..., n]).
    deltas : list[float]
```

```
        Step sizes Deltax_k = x_{k+1} - x_k (often useful for convergence diagnostics).
    final_x : float
        Final iterate x_* returned by the method (last element of xs).
    converged : bool
        Flag indicating whether the stopping criterion was satisfied.
    """

    fs: list[float]
    xs: list[float]
    ns: list[int]
    deltas: list[float]
    final_x: float
    converged: bool

@dataclass
class NewtonSystemResult:
    """
    Result container for Newton's method applied to systems of nonlinear equations.

    Parameters
    ----------
    xs : list[np.ndarray]
        Sequence of iterates x_k  in  R^n at each step (shape (n,) per iterate).
    Fs : list[np.ndarray]
        Sequence of residual vectors F(x_k)  in  R^n at each iteration.
    final_x : np.ndarray
        Final iterate x_*  in  R^n returned by the method (last element of xs).
    ns : list[int]
        Iteration indices corresponding to each x_k (typically [0, 1, ..., n]).
    """

    xs: list[np.ndarray]  # iterates x_k
    Fs: list[np.ndarray]  # F(x_k)
    final_x: np.ndarray  # x_* at termination
    ns: list[int]  # iteration indices

@dataclass
class FixedPointResult:
    """
    Result container for fixed-point iteration x_{k+1} = g(x_k).

    Parameters
    ----------
    xs : list[float]
        Sequence of iterates x_k produced by the fixed-point iteration.
    ns : list[int]
        Iteration indices corresponding to each x_k (typically [0, 1, ..., n]).
    deltaX : list[float]
        Step sizes Deltax_k = x_{k+1} - x_k, useful for monitoring convergence.
    """

    xs: list[float]
    ns: list[int]
    deltaX: list[float]
```

```python

@dataclass
class EulerResult:
    """
    Result container for Forward Euler method solutions.

    Parameters
    ------
    t : np.ndarray
        Time points where the solution is evaluated, shape (n_steps,).
    y : np.ndarray
        Numerical solution. Shape (n_steps,) for scalar ODEs or (n_vars, n_steps)
        for systems of ODEs.
    e_i : np.ndarray
        Pointwise absolute error |y_exact(t) - y_euler(t)|. Same shape as y.
        Filled with zeros if no exact solution is provided.
    y_exact : np.ndarray | None
        Exact solution evaluated at time points t (if available). Same shape as y.
        None otherwise.
    """

    t: np.ndarray
    y: np.ndarray
    e_i: np.ndarray
    y_exact: np.ndarray = None
```

Methods

Listing 5: Methods

```python
from typing import Callable

import numpy as np

from .structures import (
    EulerResult,
    FixedPointResult,
    NewtonRaphsonResult,
    NewtonSystemResult,
)

def NewtonRaphson(
    f: Callable[[float], float],
    f_prime: Callable[[float], float],
    x0: float,
    tolerance: float,
    max_iterations: int,
) -> NewtonRaphsonResult:
    """
    Newton-Raphson method to find a root of a scalar function f starting from x0.

    Parameters
    ----------
    f : Callable[[float], float]
        Scalar function f(x).
    f_prime : Callable[[float], float]
        Derivative f'(x) of the function f.
```

```
    x0 : float
        Initial guess for the root.
    tolerance : float
        Convergence tolerance on |f(x_k)|.
    max_iterations : int
        Maximum allowed number of iterations.

    Returns
    -------
    NewtonRaphsonResult
        Result container with iterates x_k, residuals f(x_k), step sizes Deltax_k,
        iteration indices, final iterate, and convergence flag.
    """
    ns: list[int] = []
    fs: list[float] = []
    xs: list[float] = []
    deltas: list[float] = []

    x = x0
    n = 0
    fx = f(x)

    xs.append(x)
    fs.append(fx)
    deltas.append(float("inf"))  # placeholder for initial step
    ns.append(n)

    converged = abs(fx) <= tolerance

    while abs(fx) > tolerance and n < max_iterations:
        n += 1
        df = f_prime(x)
        if df == 0:
            # Derivative zero -> Newton step undefined
            break

        x_new = x - fx / df
        delta_x = float(np.abs(x_new - x))
        x = x_new
        fx = f(x)

        xs.append(x)
        fs.append(fx)
        deltas.append(delta_x)
        ns.append(n)

    converged = abs(fx) <= tolerance

    return NewtonRaphsonResult(
        fs=fs,
        xs=xs,
        deltas=deltas,
        ns=ns,
        final_x=x,
        converged=converged,
    )

def fixed_point_iteration(
    g: Callable[[float], float],
    x0: float,
```

```
    tolerance: float,
    max_iterations: int,
) -> FixedPointResult:
    """
    Fixed-point iteration to solve x = g(x) starting from x0.

    Parameters
    ----------
    g : Callable[[float], float]
        Iteration function g(x) whose fixed point is sought.
    x0 : float
        Initial guess for the fixed point.
    tolerance : float
        Convergence tolerance on the step size |x_{k+1} - x_k|.
    max_iterations : int
        Maximum allowed number of iterations.

    Returns
    -------
    FixedPointResult
        Result container with iterates x_k, iteration indices, and step sizes Deltax_k.

    Raises
    ------
    RuntimeError
        If the method does not converge within max_iterations.
    """
    xs: list[float] = []
    ns: list[int] = []
    deltaX: list[float] = []

    x = x0
    DeltaX = float("inf")
    n = 0

    # Log initial state
    xs.append(x)
    ns.append(n)
    deltaX.append(DeltaX)

    while DeltaX > tolerance and n < max_iterations:
        x_old = x
        x = g(x_old)
        DeltaX = float(np.abs(x - x_old))
        n += 1

        xs.append(x)
        ns.append(n)
        deltaX.append(DeltaX)

    if n >= max_iterations and DeltaX > tolerance:
        raise RuntimeError(
            "Fixed point iteration did not converge within max_iterations."
        )

    return FixedPointResult(xs=xs, ns=ns, deltaX=deltaX)

def euler_forward(
    f: Callable,
    y0: float | np.ndarray,
```

```python
    domain: list[float],
    h: float,
    *f_args,
    exact: Callable = None,  # type: ignore
) -> EulerResult:
    """
    Solve ODE(s) y' = f(t, y) using the Forward Euler method.

    Supports both scalar ODEs and systems of ODEs.

    Parameters
    ----------
    f : Callable
        Right-hand side of the ODE:
            Scalar: f(t: float, y: float) -> float
            Vector: f(t: float, y: np.ndarray, *f_args) -> np.ndarray
    y0 : float | np.ndarray
        Initial condition. Scalar float or vector np.ndarray([y1_0, ..., yN_0]).
    domain : list[float]
        Time interval [t_start, t_end].
    h : float
        Time step size.
    *f_args
        Additional arguments passed to f(t, y, *f_args) for vector ODEs.
    exact : Callable, optional
        Exact solution y_exact(t). Called as exact(t) with t as np.ndarray.
        Should return an array of the same shape as y.

    Returns
    -------
    EulerResult
        Container with time grid t, numerical solution y, pointwise error e_i,
        and exact solution y_exact (if provided).
    """
    t = np.arange(domain[0], domain[1] + h, h)
    is_scalar: bool = np.isscalar(y0)

    n_vars: int = 1 if is_scalar else len(y0)  # type: ignore
    y: np.ndarray = np.zeros((len(t), n_vars))
    y[0] = y0 if is_scalar else y0.copy()  # type: ignore

    # Time stepping
    for i in range(len(t) - 1):
        f_val = f(t[i], y[i], *f_args) if f_args else f(t[i], y[i])
        y[i + 1] = y[i] + h * f_val

    # Shape output
    if is_scalar:
        y = y[:, 0]  # 1D array for scalar case

    # Error
    if exact is not None:
        y_exact = exact(t)
        e_i = np.abs(y_exact - y)
    else:
        y_exact = None
        e_i = np.zeros_like(y)

    return EulerResult(t=t, y=y, e_i=e_i, y_exact=y_exact)  # type: ignore
```

```python

def trapets(
    f: Callable[[float], float],
    n: int,
    boundaries: np.ndarray | list[float],
    return_h: bool = False,
) -> float | tuple[float, float]:
    """
    Approximate int_a^b f(x) dx using the composite trapezoidal rule.

    Parameters
    ----------
    f : Callable[[float], float]
        Integrand f(x) to be integrated over [a, b].
    n : int
        Number of subintervals.
    boundaries : array_like of float
        Integration limits [a, b].
    return_h : bool, optional
        If True, also return the step size h.

    Returns
    -------
    float or (float, float)
        Approximation of the integral of f from a to b.
        If return_h is True, returns (integral, h).
    """
    if n <= 0:
        raise ValueError("n has to be a positive integer")

    a, b = boundaries
    h = (b - a) / n

    x = np.linspace(a, b, n + 1)
    y = f(x)

    result = h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])

    if return_h:
        return result, h
    else:
        return result

def newton_system(
    F_func: Callable,
    JF_func: Callable,
    x0: np.ndarray,
    tol: float,
    max_iter: int,
    args: tuple,
    jac_args: tuple | None = None,
) -> NewtonSystemResult:
    """
    Newton's method for systems of nonlinear equations.

    At each iteration k, solves the linear system
        JF(x_k) * delta = -F(x_k)
    for the correction delta, then updates
        x_{k+1} = x_k + delta,
    until ||F(x_k)||_2 <= tol or the iteration limit is reached.

```

```
    Parameters
    ----------
    F_func : Callable
        Vector-valued function F(x, *args) returning np.ndarray of shape (n,).
    JF_func : Callable
        Jacobian function JF(x, *jac_args) returning np.ndarray of shape (n, n).
    x0 : np.ndarray
        Initial guess vector x_0  in  R^n (shape (n,)).
    tol : float
        Convergence tolerance on the 2-norm ||F(x_k)||_2.
    max_iter : int
        Maximum allowed number of iterations.
    args : tuple
        Additional parameters passed to F_func.
    jac_args : tuple, optional
        Additional parameters passed to JF_func. Defaults to args if not provided.

    Returns
    -------
    NewtonSystemResult
    """
    jac_args = jac_args if jac_args is not None else args

    x = x0.astype(float)
    xs = [x.copy()]
    Fx = F_func(x, *args)
    Fs = [Fx]
    ns = [0]

    for k in range(max_iter):
        if np.linalg.norm(Fx, ord=2) <= tol:
            break

        J = JF_func(x, *jac_args)
        delta = np.linalg.solve(J, -Fx)
        x = x + delta

        Fx = F_func(x, *args)

        xs.append(x.copy())
        Fs.append(Fx)
        ns.append(k + 1)

    return NewtonSystemResult(xs=xs, Fs=Fs, final_x=x, ns=ns)
```